PILOTO SDD · HARNESS RSC · INFORME DE CICLO

Feature 001 — «Exportar CSV» qué se obtuvo en cada paso del flujo

Del prompt del usuario («exportar CSV») al PR #1 mergeado: spec, plan, tareas, implementación, revisión adversarial, verificación y ship — con el artefacto concreto que produjo cada paso.

001	~66 min	17/17	1
FEATURE	CICLO COMPLETO	TESTS EN VERDE	INTERVENCIÓN HUMANA

Proyecto: Piloto SDD (cuentas personales — Node.js + Express 5 + SQLite + Vitest). **Fecha del ciclo:** 2026-11-06, 21:56 → 23:32 UTC. **Modo:** autopilot — el dueño autorizó la corrida completa sin pausas por fase; su única intervención fue elegir la feature entre 4 opciones.

El flujo que recorrió este pedido

PROMPT 21:56 → SPEC ~2 min → PLAN ~5 min → TASKS ~3.5 min → IMPLEMENT ~7 min → REVIEW ~26 min → VERIFY ~2.5 min → SHIP ~20 min

Resultado del ciclo: PR #1 «✨ Feature 001 — Exportar movimientos a CSV» MERGED el 2026-11-06 23:26:54 UTC. Suite 17/17 en verde. Revisión adversarial en 2 rondas: 6 hallazgos reportados, 6 corregidos, 0 abiertos, 0 refutados (0% de falsos positivos). Verificación final: **PASS** sobre los 5 criterios de aceptación, con el archivo abierto en Excel real.

Fuentes de este informe (artefactos del ciclo, no memoria de conversación): `02-DOCS/wiki/sdd/specs/001-exportar-csv.md` · `plans/001-exportar-csv.md` · `reviews/001-exportar-csv.md` · `features/001-exportar-csv.md` (documento único de la feature con métricas por paso).

0 El prompt del usuario — dónde empieza todo

2026-11-06 21:56 UTC · ~2 min · 1 intervención humana (la única del ciclo): elegir «exportar CSV» entre 4 opciones de feature candidatas.

El pedido entró como pedido de negocio, no como ticket técnico:

> *«Poder descargar los movimientos registrados como un archivo CSV para abrirlo en Excel. Desde la página, un botón "Exportar CSV" genera y descarga el archivo con todos los movimientos (columnas: fecha, tipo, monto, categoría, descripción). Alcance inicial: todos los movimientos; el filtrado por mes queda fuera.»*

Lo que este paso **obtiene**: el registro de ingreso en el documento de la feature — fecha, autor, descripción y contexto (primera feature del piloto, corre el flujo SDD completo para medir tiempos, tokens e intervenciones humanas). Nada de código: el documento se abre **antes** de desarrollar.

1 Spec — las historias de usuario y el contrato

~2 min · 0 intervenciones · artefacto: `specs/001-exportar-csv.md` · spec gate: PASS (4 puntos abiertos, los 4 tipados como suposición/decisión diferida/área no formulable)

Historias de usuario

HU1 — EXPORTAR TODO

Como dueño de las cuentas (usuario único, app local), **quiero** pulsar un botón «Exportar CSV» junto a la lista de movimientos, **para** descargar todos los movimientos en un solo archivo que Excel abra directamente, sin importar nada a mano.

HU2 — COLUMNAS Y ORDEN

Como dueño, **quiero** que el archivo traiga una fila de encabezados (`fecha,tipo,monto,categoria,descripcion`) y una fila por movimiento en el mismo orden que veo en la tabla (más reciente primero), **para** que el archivo no me sorprenda.

HU3 — CASOS DIFÍCILES, SIN SORPRESAS

Como dueño, **quiero** que las descripciones con comas o comillas queden en una sola celda, que los acentos y «ñ» se vean correctos y que, sin movimientos, el archivo traiga solo los encabezados, **para** poder analizarlo en Excel sin arreglos a mano.

HU4 — SIN FALLOS SILENCIOSOS

Como dueño, **quiero** que si la app no puede generar el archivo, la página muestre un error visible y no descargue nada, **para** no creer que tengo un archivo válido cuando no lo hay.

Criterios de aceptación (los 5, Given/When/Then)

AC	Criterio
AC1	Con movimientos registrados, al pulsar «Exportar CSV» el navegador descarga un `.csv` con encabezado `fecha,tipo,monto,categoria,descripcion` y una fila por movimiento, ordenadas como la tabla.
AC2	Con cero movimientos, el archivo contiene únicamente la fila de encabezados.
AC3	Una descripción con comas o comillas dobles va entrecomillada (comillas internas escapadas), de modo que Excel la muestra en una sola celda.
AC4	Categorías con acentos o «ñ» se ven correctas al abrir en Excel (UTF-8 con BOM).
AC5	Si la app no puede generar el archivo, la página muestra un error visible y no se descarga nada.

Qué quedó fuera a propósito (out of scope)

Filtro por mes/periodo (diferido) · otros formatos (XLSX, PDF) · exportar resúmenes/saldo · elegir o renombrar columnas · exportaciones programadas. El agente no puede añadir ninguno «por si acaso».

Suposiciones registradas (transparentes, no ocultas)

- Archivo en UTF-8 con BOM y separador coma (base: Excel en Windows lo abre directo; riesgo: configuración regional con «;» — cambio menor si aparece).
- Orden de filas idéntico al de la tabla: `fecha DESC, id DESC`.

2 Plan — la decisión técnica y sus contratos

> **~5 min · 0 intervenciones · artefacto:** `plans/001-exportar-csv.md` · decisiones nuevas registradas en `decisions.md`

La decisión arquitectónica principal

Generar el CSV **en el servidor** (ruta nueva `GET /api/movimientos/csv`) en vez de armarlo en el navegador a partir del listado JSON. Motivo: un solo **generador puro** (`src/csv.js`), testeable unitariamente y reutilizable; el contrato vive en un lugar y el navegador solo descarga. Trade-off aceptado: una ruta más, a cambio de testabilidad del contrato.

Componentes y responsabilidades (una por caja)

Componente	Tipo	Única responsabilidad
`src/csv.js` — generador	interno, puro	Convertir filas en texto CSV válido: BOM, CRLF, entrecomillado RFC 4180, anti-formula-injection. No toca la db ni el reloj.
Ruta `GET /api/movimientos/csv`	interno	Pedir las filas en el orden de la tabla, delegar en el generador, responder `text/csv` como attachment.
Botón «Exportar CSV» + descarga	interno	Disparar la descarga (fetch → blob → `<a download>`) y mostrar el error si falla.

Contratos definidos

- `csv.generarCsv(movimientos) → string` — BOM + encabezado + una línea CRLF por movimiento; misma entrada → misma salida.
- `GET /api/movimientos/csv → 200 text/csv` — `Content-Disposition: attachment; filename="movimientos-<YYYYMMDD>.csv"`; 200 aunque no haya movimientos; fallo → 500 con `{"errores":[...]}`.
- `descargarCsv()` — en fallo: mensaje visible, no descarga nada.

Restricciones globales que moldean todo

UTF-8 con BOM · coma · CRLF (RFC 4180) · celdas que empiezan con `= + - @` se prefijan con `'` (formula injection) · sin cambios de esquema ni migraciones · UI operable por teclado y modo oscuro intactos (constitución §8) · los datos no salen del equipo.

Riesgos identificados antes de codificar

Riesgo	Mitigación decidida en el plan
Excel regional espera «;» → todo en una columna	BOM + coma estándar; si aparece, el separador es un cambio de una línea.
Decimales con punto leídos como texto en es-PE	Mantener punto (estándar CSV); se verifica en la prueba manual y se anota.
Formula injection (`=cmd...` en descripción)	Prefijo `'` en el generador; test unit lo cubre.

3 Tasks — el desglose ejecutable

> **~3.5 min · 0 intervenciones · artefacto:** tabla de tareas anexa al plan (§8) · riesgo del conjunto medido con `rsc risk`: tier 1 — 1 revisor de contexto fresco, sin aprobación humana obligatoria

ID	Tarea	Done-check (literal, observable)	Traza
T001	Tests unit del generador CSV (`test/csv.test.js`)	`vitest run` → rojo con `src/csv.js` ausente; casos: encabezado exacto, CRLF, BOM, RFC 4180, formula injection, vacío	AC3, AC4
T002	Implementar `src/csv.js` hasta verde	`vitest run test/csv.test.js` → verde	AC3, AC4
T003	Tests integration de la ruta (`test/csv-api.test.js`)	Rojo sin la ruta; casos: body exacto con datos sembrados (orden fecha DESC), vacío → solo encabezado, headers + BOM	AC1, AC2
T004	Implementar `GET /api/movimientos/csv` hasta verde	`vitest run test/csv-api.test.js` → verde	AC1, AC2
T005	Botón «Exportar CSV» + manejo de error en `public/index.html`	El HTML contiene «Exportar CSV» Y la descarga funciona Y con servidor caído muestra error visible	AC5
T006	Verificación manual navegador + Excel, registrada	Doc de feature §Verificación con resultado: descarga OK, columnas correctas, acentos OK, decimal OK	AC4, AC5

Cobertura completa y bidireccional: AC1 (T003/T004) · AC2 (T003/T004) · AC3 (T001/T002) · AC4 (T001/T002/T006) · AC5 (T005/T006). Sin tareas sin criterio; sin criterio sin tarea.

4 Implementación — TDD rojo → verde

> **~7 min · 0 intervenciones · rama:** `feature/001-exportar-csv` · commit `7868fe2` (✨ código + tests) · suite al cierre: 16/16 en verde

Cada par test→código se ejecutó rojo primero (un test que nunca falló no prueba nada) y verde después:

- **T001+T002:** `test/csv.test.js` (6 tests: BOM, CRLF, encabezado exacto, RFC 4180, formula injection, vacío, acentos) → `src/csv.js` con `generarCsv` puro.
- **T003+T004:** `test/csv-api.test.js` (3 tests: vacío → solo encabezado, body exacto orden `fecha DESC, id DESC`, bytes `EF BB BF`) → ruta en `src/app.js` (attachment con fecha del día, 500 con `{"errores":[...]}` si falla).
- **T005:** botón en la sección Movimientos: fetch → blob → `<a download>`, error visible en `#error-exportar`, `<button type="button">` operable por teclado, estilos con variables CSS (modo oscuro intacto).

Verificado contra la **app real** levantada en localhost:3000: el botón está en el HTML y la ruta responde 200 con BOM, `text/csv; charset=utf-8` y `Content-Disposition` con la fecha del día.

Decisiones registradas: tests decodifican con `ignoreBOM: true` (el BOM viaja en los bytes, verificado con `arrayBuffer`); nombre de archivo tomado del `Content-Disposition` con fallback; sanitización anti-injection solo en el generador.

5 Revisión adversarial — el paso que encontró lo que los tests no veían

> **~26 min · 0 intervenciones · 4 subagentes** (revisor r1, 2 finding-verifiers, revisor r2; 134,940 tokens) · registro: `reviews/001-exportar-csv.md` + ledger local

Revisor de **contexto fresco** — no vio la conversación de implementación — y cada hallazgo severo pasó por el **finding-verifier**, cuyo trabajo era demostrarlo falso. Solo lo reproducido sobrevive:

ID	Severidad	Hallazgo	Verificación	Outcome
L1	blocker	El error de exportación **nunca podía verse**: la clase CSS `.error` fijaba `display:none` y el `style.display=''` del catch no la vencía → AC5 incumplido.	CONFIRMED — reproducido en Edge headless: `display=none, offsetHeight=0`	fixed
L2	should-fix	El guard de formula injection retornaba antes del entrecomillado: `=SUM(A1), total` salía sin comillas → 6 columnas.	CONFIRMED — reproducido en Node: la fila parsea a 6 campos	fixed
L3–L5	3 nits	BOM como literal invisible en el fuente · `catch` sin log server-side · fila duplicada en el doc de métricas.	aceptados	fixed

Ronda 1: CHANGES REQUESTED (16/16 verde igualmente — verde no era suficiente). **Fixes** (commit `a3ff919`): visibilidad del error gobernada solo con inline (verificado en navegador real: `display=block, offsetHeight=18`, mensaje visible); el prefijo anti-injection se aplica y *luego* se evalúa el entrecomillado, con test nuevo que afirma 5 columnas tras parseo RFC 4180.

Ronda 2 (solo la divergencia de fixes, revisor nuevo): 1 nit (literal BOM inconsistente en el test), corregido. **Veredicto final:** APPROVE WITH NITS — 6/6 fixed, 0 abiertos, 0 refutados, falso-positivo 0%.

> **Lección para la próxima feature con UI:** `curl` no evidencia el render del navegador — el blocker de UI solo era detectable con navegador real (headless), no con curl.

6 Verificación — evidencia, no promesas

> **~2.5 min · 0 intervenciones · resultado:** PASS — los 5 criterios con evidencia ejecutable

Control (DoD de la constitución)	Evidencia observada
Suite en verde con los tests nuevos	`npm test` → **17/17** (4 archivos). Literal: «Test Files 4 passed (4) · Tests 17 passed (17)».
Archivo real contra la app viva	Descargado de localhost:3000 y parseado con parser RFC 4180: BOM `EF BB BF` ✓ · encabezado exacto ✓ · todas las filas con 5 columnas ✓ · «Alimentación» íntegro ✓ · `75.5` tal cual ✓.
Excel real (COM automation)	El archivo se abrió en Excel: **5 columnas, 3 filas**; A1 = «fecha», D2 = «Alimentación» (acento íntegro), C2 = `75.5` leído como número. T006 cumplido sin abrir Excel a mano.
Camino de error (AC5)	Edge headless pulsando el botón con fetch fallido → error visible (`display=block, offsetHeight=18`), sin descarga.
Sin secretos ni datos en el diff	`git diff main...HEAD` escaneado: única coincidencia es el `DATA_DIR` temporal de los tests (esperado); `data/` no viaja.
Revisión sin bloqueantes	APPROVE WITH NITS, 6/6 hallazgos fixed, 0 abiertos.

Nota honesta: la «verificación manual» de T006 la ejecutó automatización (Edge headless + Excel COM) por el autopilot; el dueño puede repetirla abriendo http://localhost:3000 y pulsando «Exportar CSV».

7 Ship — cierre del ciclo

> **~20 min** (≈15 por dos cortes transitorios del clasificador de permisos del harness de agentes — no del proyecto; resueltos esperando y reintentando) · **0 intervenciones**

- **PR #1** «✨ Feature 001 — Exportar movimientos a CSV» — MERGED el 2026-11-06 23:26:54 UTC (merge commit, historial por fases preservado).
- Auto-merge autorizado por el dueño, condicionado a: verificación PASS + 0 hallazgos bloqueantes. Ambas condiciones se cumplieron.
- Main actualizado y empujado (push autorizado expresamente para esta corrida). Worklog del ciclo: `02-DOCS/raw/worklog/2026-11-06-001-exportar-csv.md`.

Métricas del ciclo completo (por paso)

Paso	Duración	Tokens (in / caché / out)	Interv. humanas	Nota
Registro (prompt)	~2 min	—	1	Feature elegida entre 4 opciones
Spec	~2 min	28,413 / 1,452,480 / 8,939	0	Gate PASS; 4 puntos tipados
Plan	~5 min	25,813 / 3,564,480 / 17,174	0	CSV en servidor; anti-injection desde el plan
Tasks	~3.5 min	92,792 / 1,398,784 / 6,122	0	6 tareas; riesgo tier 1
Desarrollo	~7 min	124,072 / 3,209,472 / 19,747	0	TDD rojo→verde; 16/16; app real
Revisión	~26 min	126,511 / 9,393,408 / 56,007 · subagentes 134,940	0	2 rondas; 6 hallazgos, 6 fixed
Verificación	~2.5 min	7,576 / 1,181,312 / 8,008	0	PASS: 17/17, RFC 4180, Excel real
Ship	~20 min	27,001 / 5,886,528 / 15,643	0	PR #1 merged; main empujado
Total	**~66 min efectivos**	**432,178 / 26,086,464 / 131,640 · subagentes 134,940**	**1**	Primer ciclo SDD completo

> **Qué demostró este ciclo:** el flujo completo del harness funciona de punta a punta en una feature real — y el paso más caro fue exactamente el que debía serlo: la revisión adversarial (26 de 66 min) fue la única capa que detectó el blocker de UI que 16 tests verdes no vieron. La fricción se concentró donde el riesgo vivía.

Costo en dinero: n/d hasta tener la tabla de precios GLM. Tokens de subagentes reportados por el harness sin desglose in/out.